

Data-frames and Datasets

Sure! Let's dive into the concept of Datasets in Spark.

In Spark, a dataset is like a well-organized library of books, where each book represents a piece of data. Just like how each book has a specific genre and author, datasets consist of strongly typed objects, meaning each piece of data has a defined type (like a number or a string). This makes it easier to work with the data because you know exactly what type of information you're dealing with, which helps prevent mistakes.

An Engaging Example

Imagine you are sorting your collection of recipes. Each recipe has a specific type, like "dessert" or "main course," and you want to ensure that you only use the right ingredients for each type. Datasets work similarly by ensuring that the data is organized and type-safe, allowing you to perform operations like adding, averaging, or grouping data efficiently. This organization helps Spark understand the data better, leading to faster computations and optimized memory usage.

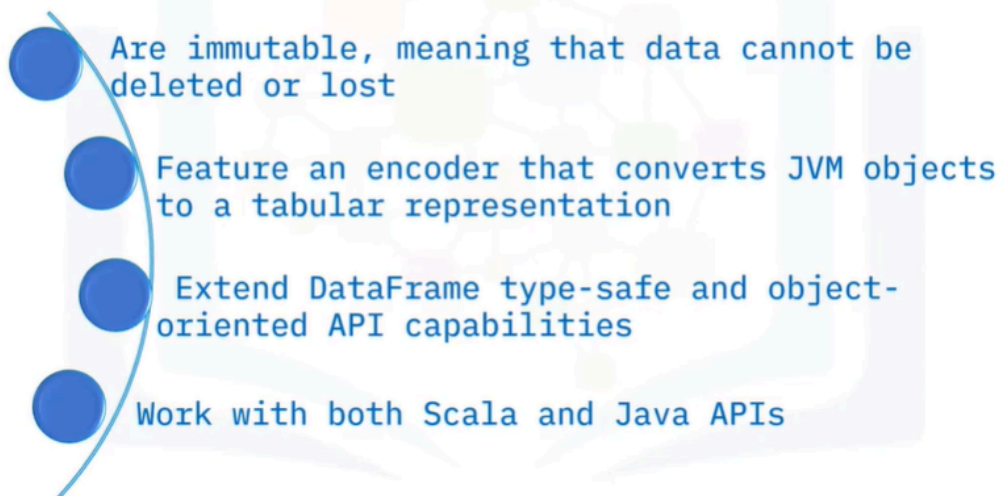
Datasets

A dataset is a distributed collection of data that:

- Consists of a collection of strongly typed JVM objects
- Provides the combined benefits of both RDDs and Spark SQL

- Datasets are the newest Spark data abstraction. Like RDDs and DataFrames, datasets provide an API to access a distributed data collection. Datasets are a collection of strongly typed Java Virtual Machine, or JVM, objects. Strongly typed means that datasets are typesafe, and the dataset's datatype is made explicit during its creation. Datasets provide the benefits of both RDDs, such as lambda functions, type-safety, and SQL Optimizations from SparkSQL.

Datasets Features



- Speaking of benefits, here are four more features of datasets: Datasets are immutable. Like RDDs, they cannot be deleted or lost. Datasets feature an encoder that converts type-specified JVM objects to a tabular representation. Datasets extend the DataFrame API. Conceptually a dataset of a generic untyped "Row" is a JVM object seen as a column of a DataFrame. Because datasets are strongly typed, APIs are currently only available in Scala and Java, which are statically typed languages. Dynamically typed languages, such as Python and R, do not support dataset APIs.

Datasets in Spark - Benefits

- Provide compile-time type safety
 - Compute faster than RDDs
 - Offer the benefits of Spark SQL and DataFrames
 - Optimize queries using Catalyst and Tungsten
 - Enable improved memory usage and caching
 - Use dataset API functions for aggregate operations including sum, average, join and group by
-
- Datasets offer some unique advantages and benefits over using DataFrames and RDDs. Because Datasets are statically-typed, datasets provide compile-time type safety. Compile-time type safety means that Spark can detect syntax and semantic errors in production applications before deployment, saving substantial developer and operational costs and time. Datasets compute faster than RDDs, especially for aggregate queries. Datasets offer the additional query optimization enabled by Catalyst and Tungsten. Datasets enable improved memory usage and caching. But how? Spark understands the data structure of the datasets and optimizes the layout within memory. The dataset API also offers functions for convenient high-level aggregate operations, including sum, average, join, and "group-by."

Creating a dataset

Apply the `toDS()` function to create a dataset from a sequence

```
// Create a Dataset from a sequence of primitive datatype  
val ds = Seq("Alpha", "Beta", "Gamma").toDS()
```

- Let's now look at three ways that you can create a dataset within Spark. This first example, written in Scala, uses the `toDS` function to create a dataset from a sequence.

Create a dataset from a text file

Create a dataset from a text file

```
// Create a Dataset from a file for a primitive datatype  
val ds =  
spark.read.text("/text_folder/file.txt").as[String]
```

- This second example illustrates how to create a dataset from a text file. Notice the explicit schema declaration. In this example, we apply the primitive "String" data type to the explicit schema declaration.

Create a Dataset from a JSON file

Create a dataset using a JSON file

```
// Create a Dataset from a file for a custom datatype
case class Customer(name: String, id: Int, phone: Double)
val ds_cust =
  spark.read.json("/customer.json").as[Customer]
```

- This third and final example creates a dataset using a JSON file. In this last example, we use a custom class called "Customer," which contains a "name" and an "ID" field.

Datasets & DataFrames Compared

Datasets are:

- Strongly-typed
- Use unified Java and Scala APIs
- Built on top of DataFrames and the latest data abstraction added to Spark

DataFrames are:

- Not typesafe
- Use APIs in Java, Scala, Python and R
- Built on top of RDDs and added in earlier Spark versions

- Next, let's compare datasets to DataFrames. Datasets offer the benefits of both its predecessors – DataFrames and RDDs. Datasets are strongly typed. DataFrames are not typesafe. Datasets can use unified Java and Scala APIs. DataFrames use APIs that are written in Java, Scala, Python, and R. The

DataFrames API may or may not be unified depending on the API version. Datasets are the latest addition to Spark and are built on top of DataFrames. In contrast, DataFrames, introduced earlier, are built on RDDs.

Summary

In this video, you learned that:

- A dataset is a distributed collection of data that provides the combined benefits of both RDDs and Spark SQL
- Datasets consist of strongly typed JVM objects
- Datasets use of DataFrame typesafe capabilities and extend object-oriented API capabilities
- Datasets work with both Scala and Java APIs